The data of Pollis, and yours beside it

PollisDatasets.jl, PollisJournals.jl, PollisReferences.jl and PollisRSS.jl: what each holds, how it reaches your computer and becomes a database, how it is updated without a new Pollis, and how the data you add lives beside it without ever being touched.

Drill-down. A look under the surface of what the Guide describes in *Data and Databases* and *Built-in Databases*. It is for anyone who wants to know where their data is and why it is safe, and for whoever maintains Pollis (§10 is for them). It describes the state of 25 October 2026.

Contents

1 The data layer in one table 2

2 The principle: Pollis's data is replaceable, yours never 2

3 The whole flow at a glance 3

4 PollisDB.jl: the Julia side of Pollis 3

5 The four packages 3

5.1 PollisDatasets.jl: the datasets of the examples . . . 3

5.2 PollisJournals.jl: every journal, and the Norwegian register . . . 4

5.3 PollisReferences.jl: every work Pollis cites . . . 4

5.4 PollisRSS.jl: the feeds Pollis offers . . . 5

6 The install road: from GitHub to four databases 5

6.1 When it runs . . . 5

6.2 Installing and updating . . . 5

6.3 Copying . . . 5

6.4 Building . . . 5

7 The build road: PollisReferences and PollisRSS inside Pollis 6

7.1 Pinning . . . 6

7.2 Downloading . . . 6

7.3 What `compile-toml` makes . . . 6

8 Run time: which copy wins 6

8.1 References . . . 6

8.2 RSS catalogue . . . 7

8.3 Datasets and journals . . . 7

9 Your data beside Pollis's 7

9.1 Your own databases . . . 7

9.2 Journal metrics you import . . . 7

9.3 References you save, correct and merge . . . 7

9.4 Feeds you follow, and what they bring . . . 8

9.5 Why it holds together . . . 8

9.6 Back up, move, start again . . . 8
9.7 Giving your data to everyone . . . 8

10 Procedures, for maintainers **8**
10.1 Add a dataset . . . 9
10.2 A month of PollisJournals . . . 9
10.3 Correct a work, or add one . . . 9
10.4 Cite a new work in a panel . . . 9
10.5 Add or correct a feed . . . 9
10.6 Check a change end to end . . . 9

11 Where things are **10**

12 Open points **10**

1 The data layer in one table

Pollis is a program; the data it comes with is not part of it. Four Julia packages on GitHub hold that data, each changing on its own timetable, so a new dataset, a new month of journals, a corrected DOI or a new feed reaches you without a new Pollis.

	PollisDatasets.jl	PollisJournals.jl	PollisReferences.jl	PollisRSS.jl
Holds	57 real datasets, each with its source, licence and columns	251,461 journals, conferences and series; the Norwegian register	650 works Pollis cites, 797 dated citation counts	519 feeds under 35 topics
Form	CSV files and `datasets.toml`	Parquet files in a Julia artifact	one JSON file	one JSON file
Changes	when a dataset is added	monthly, by a workflow	when a work is corrected or cited	when a feed is added or corrected
Database	PollisDatasets	PollisJournals	PollisReferences	PollisRSS
In Julia	`pollis`	`journals`	`references`	`rss`
Reaches Pollis	install road	install road	install and build roads	install and build roads
Licence	code MIT; each dataset its own	code MIT; data CC0 and NLOD 2.0	MIT	MIT

All four live under `github.com/Trumpingtons`, all are installed by URL, not from the General registry, and Pollis follows the latest *commit* of their `main`, not version numbers (which are chosen by hand; Pollis does not use semver).

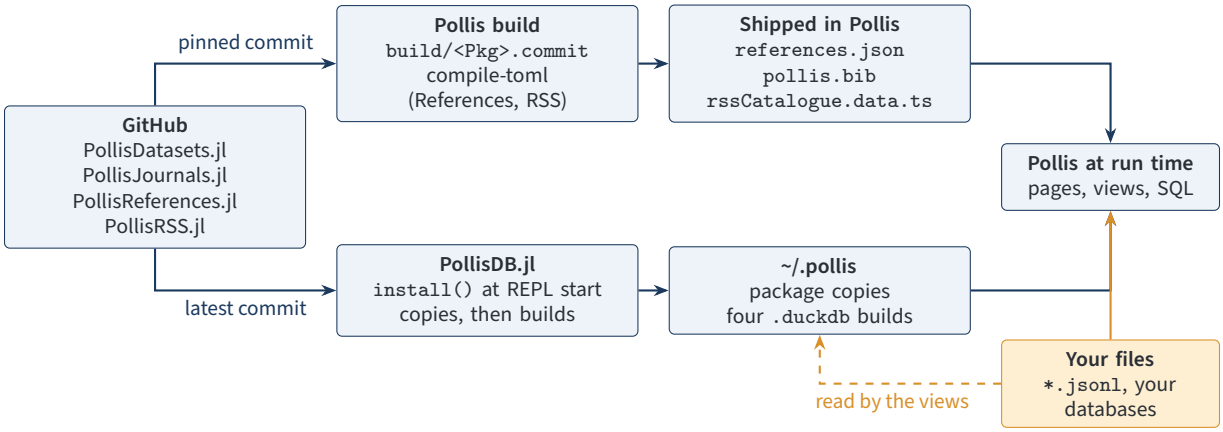
2 The principle: Pollis's data is replaceable, yours never

Three rules, and everything else follows from them.

1. **What a package brings is a copy that an update may replace.** Pollis never asks you to look after it.
2. **What you do is written to files of yours that no update writes.** They are JSON Lines, appended to and never rewritten: a change is a new line, and for each key *the last line wins*.
3. **The four built-in databases are builds over both**, read-only, rebuilt when what they come from changes. Deleting one loses nothing: it is made again at the next start of the Julia REPL.

Pollis's, replaced by updates	Yours, never replaced
the installed packages, in `~/.pollis/julia/environments/v1.13`	your own databases, wherever you keep them
`~/.pollis/databases/*.duckdb` and their `.stamp` files	`~/.pollis/journals/metrics/*.jsonl`, the metrics you imported
`~/.pollis/databases/PollisDatasets.json`	`~/.pollis/journals/articles.jsonl`, articles looked up
`~/.pollis/journals/scores.csv`	`~/.pollis/references/references.jsonl`, `counts.jsonl`, `versions.jsonl`, `panels.jsonl`
`~/.pollis/references/pollis.json`, `package.json`	`~/.pollis/rss/feeds.jsonl`, `items/*.jsonl`, `zotero.jsonl`, `episodes.jsonl`
`~/.pollis/rss/catalogue.json`	the RSS settings: unsubscribed, removed, hidden, welcome page

3 The whole flow at a glance



Two roads lead from a package to you. The *install road* takes the newest commit on GitHub through the Julia REPL, copies what Pollis's pages read, and builds the databases; all four packages take it. The *build road* takes PollisReferences.jl and PollisRSS.jl at a commit pinned in the Pollis repository and bakes them into every Pollis, because the References and RSS pages must work in a Pollis without Julia, offline, or on its very first start. PollisDatasets and PollisJournals need no build road: they are only ever read through Julia and DuckDB. Your files reach the pages directly, and reach the databases through views that read them whenever they are queried.

4 PollisDB.jl: the Julia side of Pollis

The four packages are data; what installs them, builds the databases and connects you to them is `PollisDB.jl`. It is not a package: it is a Julia module inside Pollis itself, in its Julia extension (`extensions/language-julia/scripts/terminalserver/PollisDB.jl`), under Pollis's licence, AGPL-3.0. Every time the Julia REPL starts, Pollis loads it into the REPL as `PollisDB`, so it is always there, in any project, without being added to it. It has four jobs.

1. **Keep the data packages installed and current** (`install()`, `needs_install()`): it adds the four packages, and DuckDB, to an environment of Pollis's own, `~/.pollis/julia/environments/v1.13`, and updates them when GitHub has newer commits. That environment is on the load path, so `usingDuckDB` or `usingPollisDatasets` works in any of your projects without touching its `Project.toml`.
2. **Build the four databases** (`build()`, `build_journals()`, `build_references()`, `build_rss()`), and copy the data files that Pollis's pages read (`copy_package_data()`).
3. **Connect** (`connect()`, `sql()`, the `sql>` mode): a DuckDB connection with the four built-in databases attached read-only, and Lance loading in the background.
4. **Feed the Databases view** (`show()`, `register()`): each connection handed to it is described, schemas, tables and columns, in a JSON file in `~/.pollis/databases/sessions`, which the view watches.

Its DuckDB is Pollis's fork of DuckDB.jl, `github.com/Trumpingtons/DuckDB.jl`, which brings no engine of its own: it loads the one Pollis ships, copied to `~/.pollis/duckdb` with the Lance extension, so every DuckDB in Pollis, the `duckdb>` terminal included, is the same engine.

5 The four packages

5.1 PollisDatasets.jl: the datasets of the examples

What it holds. 57 small, real datasets, from `mtcars` and `iris` to `mroz` and `challenger`, grouped by domain (Regression, Time Series, …), that the Code Examples of Pollis load. Each is `data/<id>.csv` with an entry in `datasets.toml`:

```
[[datasets]]
id = "mtcars"
title = "Motor Trend Cars"
domain = "Regression"
description = "Fuel consumption and ten design and performance variables for 32 cars (1973-74 models)."
rows = 32
source = "Henderson, H. V. and Velleman, P. F. (1981), Building multiple regression models interactively, ..."
url = "R datasets package (mtcars)"
licence = "None stated; R datasets package (GPL-2 | GPL-3)"
status = "redistributed"
columns = [ { name = "model", description = "Car make and model" }, { name = "mpg", description = "Miles per US
    gallon" }, ... ]
```

Licences. Every dataset passed a licence check before it came in, and `status` records the outcome: `open` (an explicit open licence such as CC0 or CC BY, or permission given; 20 datasets), `public-domain` (a US-government work, or published before

1930; 5) or `redistributed` (no explicit licence, but a small factual table redistributed for decades in CRAN and textbook packages; 32). Datasets that failed are listed, with the reason, at the top of `datasets.toml`: the Fama-French factors and the S&P 500 (no redistribution allowed: fetch them live from **Explore › Access Data Libraries**), the SILSO sunspots (CC BY-NC, incompatible with AGPL-3.0) and Boston Housing (an ethically problematic variable).

In Julia. `dataset("mroz")` loads a dataset as a Tables.jl column table (never a DataFrame); `datasets()` lists them; `datasetinfo("mroz")` gives the description, the source to cite, the licence and the columns. The package depends on CSV, TOML and Tables only.

The database. `PollisDB.build()` makes `~/.pollis/databases/PollisDatasets.duckdb`: one *table* per dataset, named by its id, with `COMMENT ON TABLE` (title, description, source, licence) and `COMMENT ON COLUMN` from `datasets.toml`, so the descriptions show in the Databases view and in DuckDB's own catalogue. It is rebuilt when its stamp, `PollisDatasets.duckdb.stamp`, no longer matches: the stamp is the folder of the installed package (which changes with every version installed) and the latest modification time of `datasets.toml` and the CSV files (which covers a copy under development).

New datasets announced. `PollisDB.catalogue()` writes `~/.pollis/databases/PollisDatasets.json`, the id and title of every dataset. The Welcome page compares it with the ids you have already seen, and a notification names the new ones, with **Open Databases View**.

5.2 PollisJournals.jl: every journal, and the Norwegian register

What it holds. Two Parquet files in a Julia *artifact* named `journals`, not in the repository:

- `journals.parquet`: every source in OpenAlex with at least one work, 251,461 in the snapshot of 2026-10-14 (203,247 journals, and repositories, conferences, book series and ebook platforms), with ISSNs, publisher, country, field, works, citations, the 2-year mean citedness, h-index, open access, article charges and the counts of every year;
- `norwegian.parquet`: the Norwegian Register for Scientific Journals, Series and Publishers, 40,245 rows on 2026-10-20, with each title's level in every year since 2004.

How it is updated. A GitHub workflow runs on the 2nd of every month. When OpenAlex has a new snapshot of its sources, it builds both files (`build/build.py`), publishes them as a release named after the snapshot's date, and commits the new `Artifacts.toml`. Pollis sees a new commit, updates the package, and Julia downloads the new artifact. In Julia, `PollisJournals.parquet()`, `norwegian()` and `snapshot()` give the two files and the date.

Licences. OpenAlex's data is CC0; the Norwegian register is under the Norwegian Licence for Open Government Data (NLOD) 2.0, distributed by HK-dir; the package code is MIT. Impact factors, CiteScores and quartiles are *not* free to share, so the package never holds them: you import your own (§9.2).

The database. `PollisDB.build_journals()` makes `PollisJournals.duckdb`:

Name	Kind	From
`journals`	table	`journals.parquet`, without the yearly counts
`journal_years`	table	the yearly counts, a row per journal and year
`journal_issns`	table	every ISSN, print and electronic, with its journal's `id`
`norwegian_register`	table	`norwegian.parquet`, with `journal_id` found by ISSN
`norwegian_levels`	table	a row per title and year: 0, 1, 2 or X
`metrics`	view	*your* imported files, `~/.pollis/journals/metrics/*.jsonl`
`journal_metrics`	view	the same, with `journal_id` found by ISSN
`articles`	view	*your* articles looked up, `~/.pollis/journals/articles.jsonl`, the latest line of each

With it, the build writes `~/.pollis/journals/scores.csv`: for every ISSN, the latest Norwegian level, the mean citedness, the h-index, open access and DOAJ. The RSS pages read this file to tag journal items, so they need no database at run time. The stamp is the package folder plus the SQL of the views and of the scores, so a Pollis that changes them rebuilds the database too.

5.3 PollisReferences.jl: every work Pollis cites

What it holds. `data/references.json`: one object, tab-indented, UTF-8, with two arrays.

- `references`: a work each, sorted by `key`. Fields, in database order: `key`, `type`, `title`, `authors`, `author_kind`, `authors_complete`, `year`, `released`, `journal`, `booktitle`, `editors`, `publisher`, `address`, `edition`, `volume`, `number`, `pages`, `institution`, `report_number`, `isbn`, `issn`, `doi`, `arxiv`, `openalex`, `url`, `oa_url`, `abstract`, `abstract_source`, `keywords`, `keywords_source`, `legacy_key`, `source`, `retrieved`, `checked`, `open_access`. A field not known is left out; the build fills it with `null` in this order (`REFERENCE_FIELDS` in `build/lib/pollis-bib.ts`).
- `counts`: `key`, `count`, `source` (`crossref` or `openalex`), `date`; appended, never changed, never added up.

The *panels* that cite each work are not in the package: they belong to a given Pollis, so the build adds them to the copy it ships (§7). In Julia, `PollisReferences.json()` is the path of the file.

The database. `PollisDB.build_references()` makes `PollisReferences.duckdb`: the *tables* `pollis_references` and `pollis_counts` from Pollis's copy (§8), and *views* over your files (§9.3), joined in `all_references`, `all_counts` and `all_versions`.

5.4 PollisRSS.jl: the feeds Pollis offers

What it holds. `data/feeds.json`, the catalogue of **Explore › RSS › RSS Feeds**: the feeds, grouped by topic, and the few a new user follows at first. *Never the items*: those are read by your Pollis and are yours (§9.4).

```
{
  "topics": [
    { "topic": "Julia", "feeds": [
      { "name": "JuliaLang Blog", "url": "https://julialang.org/feed.xml" },
      { "name": "The Julia Programming Language",
        "url": "https://www.youtube.com/feeds/videos.xml?channel_id=UC9IuUwwE2xdjQUT_LMLONoA", "note": "youtube" } ]
    },
    { "topic": "Statistics", "feeds": [
      { "name": "Biometrika", "url": "https://academic.oup.com/rss/site_5265/3131.xml",
        "issns": ["0006-3444", "1464-3510"] } ] }
  ],
  "defaults": [ { "url": "https://julialang.org/feed.xml", "topic": "Julia" }, ... ]
}
```

- `topic`: the name stored with every feed you follow, in `feeds.jsonl`. It is not translated, and renaming it is a migration (§10.5).
- `note`: an *id*, never text: `paywall`, `busy`, `weekdays`, `preprints`, `youtube`, `podcast`, `readersLinks`, `newEntries`, `weekly`. Pollis translates the ids it knows (`catalogueNote` in `rssCatalogue.ts`) and shows nothing for one it does not. No page displays the notes yet.
- `media`: `"audio"` for a podcast's feed; a YouTube channel is recognised by its address.
- `issns`: the journal's ISSNs, by which PollisJournals finds its level and your metrics.
- `defaults`: the feeds a new user follows at first. The rules for choosing them are still to be decided.

The database. `PollisDB.build_rss()` makes `PollisRSS.duckdb`, which has no tables at all: its four *views*, `items`, `feeds`, `zotero` and `episodes`, read your files (§9.4). The database is made at install, with PollisDatasets, and its stamp is the SQL of the views alone: it is rebuilt only when a Pollis changes them, never because items arrived.

6 The install road: from GitHub to four databases

6.1 When it runs

At every start of the Julia REPL, `terminalserver.jl` asks `PollisDB.needs_install()`. It is true when a package is missing or GitHub has a newer commit, a database is missing or behind, or a package copy is behind its package. Then the Julia extension runs `install_pollisdb.jl` in a terminal of its own, so the REPL never waits for `Pkg`:

```
PollisDB.install()            # add what is missing, update what GitHub has newer
PollisDB.catalogue()          # PollisDatasets.json, for the Welcome page
PollisDB.copy_package_data()  # package.json and catalogue.json
PollisDB.build(); PollisDB.build_rss(); PollisDB.build_journals(); PollisDB.build_references()
PollisDB.warm_lance()         # DuckDB's Lance extension, loaded once
```

6.2 Installing and updating

`DATA_PACKAGES` lists the four packages, each with its GitHub URL and the file `~/.pollis/julia/environments/v1.13/<Pkg>.commit` that records the commit installed. `pending()` asks GitHub for the latest commit of `main` (the commits API, 3-second timeout):

- a package not installed is *missing*, and all missing ones are added by URL in one `Pkg.add`;
- a package whose recorded commit differs from GitHub's is *outdated*, and `Pkg.update` fetches it;
- offline, or for a copy you installed in an environment of your own, nothing is done.

The packages go into Pollis's own environment, never into your projects; your active project is left as it was.

6.3 Copying

`PACKAGE_COPIES` says which data file goes where: `references.json` to `~/.pollis/references/package.json` and `feeds.json` to `~/.pollis/rss/catalogue.json`. `copy_package_data()` copies a file only when it differs, through a temporary file and a rename, so a reader never sees half a file; a copy interrupted is finished at the next start.

6.4 Building

Every build follows the same pattern:

1. compare the *stamp* of the database with what it would be now; if equal, do nothing;
2. build the new database under a temporary name, `<name>.duckdb.<pid>.tmp`;
3. move it over the old one in a single rename and write the stamp.

A connection open on the old file keeps reading it, untroubled; no reader ever sees a half-made database. If the build fails, the temporary file is deleted and *the previous database is kept*, with a warning; only when there was none does the error stop the install.

Database	Rebuilt when
PollisDatasets	a new version of the package is installed, or its CSV files or `datasets.toml` change
PollisJournals	a new version of the package (a new snapshot) is installed, or a Pollis changes its views or scores (`journals_outdated()`)
PollisReferences	`package.json` or `pollis.json` changes, or a Pollis changes its views
PollisRSS	a Pollis changes its views; never for new items

None is rebuilt because of something you did: your data reaches them through views.

7 The build road: PollisReferences and PollisRSS inside Pollis

Only two of the four packages take this road, because only their data is read by Pollis's own pages, which must work without Julia: the References pages (Browse References, Explore References, Discover Literature) and the RSS pages. PollisDatasets and PollisJournals are read only through Julia and DuckDB, so a Pollis without Julia has no use for them, and baking 57 datasets or 250,000 journals into every Pollis would only make it bigger. (The RSS pages' journal tags come from `scores.csv`, which the install road writes.)

7.1 Pinning

`build/PollisReferences.commit` and `build/PollisRSS.commit` hold one full commit SHA each. They are the only link between a Pollis source tree and the packages: a Pollis built from a given commit of its repository always gets the same data, whatever happens on GitHub later.

7.2 Downloading

`build/lib/pollis-packages.ts`, `pinnedPackageFile(repoRoot,{name,file,clone})`:

1. if the environment variable named by `clone` (`POLLIS_REFERENCES`, `POLLIS_RSS`) is set, it is the path of a local clone, and `<clone>/data/<file>` is read as it is, committed or not;
2. else `.build/<name>/<commit>/<file>` is read, if it was downloaded before;
3. else `https://raw.githubusercontent.com/Trumpingtons/<name>.jl/<commit>/data/<file>` is fetched and cached there.

A failed download stops the build with the URL and the HTTP status.

7.3 What `compile-toml` makes

`build/gulpfile.toml.ts` runs first in `compile-client` and `watch-client`, so every build has it:

- from PollisReferences.jl: the panels' `*.data.ts` with their references expanded from keys; `pollis.bib`; `media/references/references.json`, the package's works and counts *plus* `panels`, the works each panel cites; and the citation copies in the Compose notebooks and the shipped methods. A key cited anywhere but missing from the package stops the build: *Reference keys not in PollisReferences.jl: …*
- from PollisRSS.jl: `src/vs/workbench/contrib/monitor/browser/rssCatalogue.data.ts`, which exports `SHIPPED_CATALOGUE`. Like the panels' `.data.ts`, it is generated and git-ignored.

Generated files are written only when their text changed. `build/gulpfile.vscode.ts` and `build/next/index.ts` ship `references.json` and `pollis.bib`; the RSS catalogue is compiled into the workbench code.

8 Run time: which copy wins

8.1 References

The Julia extension copies the `references.json` of the running Pollis to `~/.pollis/references/pollis.json` at activation. Two readers then apply the same rule:

> Use `package.json` (the works and counts of PollisReferences.jl) if it has *every key that the panels of this Pollis cite*; otherwise use the works and counts this Pollis was built with. The panels always come from this Pollis.

- **PollisReferences database**: `references_source(con)` applies the rule in one DuckDB query.
- **References pages** (Browse References, Explore References, Discover Literature): `ReferenceLibrary.pollis()` in `referenceLibrary.ts` applies it when first asked, once per window.

The rule makes the package safe across Pollis versions: an older Pollis whose panels cite a key that the package no longer has keeps its own references rather than show a panel with a missing work.

8.2 RSS catalogue

`rssCatalogue.ts` starts with `SHIPPED_CATALOGUE`. When the RSS service starts, it reads `~/.pollis/rss/catalogue.json` and passes it to `setRssCatalogue()`, which accepts it only if every topic has a name and a list of feeds, and every feed a name and an address; a file missing or malformed leaves the shipped catalogue. A catalogue copied while Pollis is open is taken at the next start of the window.

8.3 Datasets and journals

No choice to make: there is only the installed package. Before the first install has finished, the Databases view and the Welcome page's **Status** say that the databases are not ready yet; the journal tags of the RSS pages appear when `scores.csv` does.

9 Your data beside Pollis's

Pollis brings data; you bring more, and the two should meet without either spoiling the other. The way they meet is the same everywhere: *your data is in your files; Pollis's is in its packages; a database or a page puts them side by side when you look*. The sections below go through each kind of data you add.

9.1 Your own databases

The built-in databases are read-only, and should be: the next build would replace anything written into them. Your tables go in a database of your own, which can sit beside them.

- **Make one.** In the Databases view, **New** › **DuckDB**, and choose a file (or none, for a database in memory that reads CSV and Parquet files in place). PostgreSQL and SQLite databases connect the same way.
- **Fill it.** **Explore** › **Access Files and Databases** › **SQL Databases** imports a CSV, Parquet or JSON file and saves a query's result as a table, always into your database. From Julia, any `CREATE TABLE` on your connection.
- **Use it with Pollis's.** `PollisDB.connect("sales.duckdb")` opens your file *and attaches the four built-in databases, read-only, under their names*. The `duckdb>` terminal and the `sql>` mode do the same. One query can then join your table to Pollis's:

```
db = PollisDB.connect("mydata.duckdb")
DBInterface.execute(db, """
  CREATE TABLE my_journals AS
  SELECT m.*, j.h_index, j.mean_citedness_2yr
  FROM my_list m JOIN PollisJournals.journal_issns i ON i.issn = m.issn
  JOIN PollisJournals.journals j ON j.id = i.id""")
```

  The new table is yours, in your file; the built-in databases are untouched, and the next update of PollisJournals does not affect it. Re-run the query to bring it up to date.

DuckDB never downloads an extension behind your back in these connections (`autoinstall_known_extensions` and `autoload_known_extensions` are off): the engine and the Lance extension come with Pollis, in `~/.pollis/duckdb`.

9.2 Journal metrics you import

Import Journal Metrics..., on PollisJournals in the Databases view or on the Journal Metrics page, reads an export of SCImago, CWTS, ABDC, Scopus or Journal Citation Reports. Pollis recognises the source by the columns, and writes the rows, in one form for all sources, to a file of its own in `~/.pollis/journals/metrics`, named after the source and the file (`SCImago-scimagojr-2025.jsonl`); `~/.pollis/journals/latest` keeps, for the RSS tags, the latest year of each journal in each import. The views `metrics` and `journal_metrics` read every file there whenever they are queried, so:

- an import is in the database at once, with no rebuild;
- importing the same file again replaces its rows, and **Remove** deletes the import and its rows;
- a new month of PollisJournals rebuilds the tables, and your metrics are joined to the new journals by ISSN.

9.3 References you save, correct and merge

Every change you make to references is a line appended to a file in `~/.pollis/references`:

File	A line is
`references.jsonl`	a work you saved (Discover Literature, a panel's Explore References), or your correction of a work of Pollis's, under the same key; or `removed`, or `merged_into` another key
`counts.jsonl`	a citation count you fetched, with its source and date
`versions.jsonl`	a record of a work in OpenAlex or Semantic Scholar: the article, a preprint, a working paper
`panels.jsonl`	a work you added to, or hid from, a panel's Explore References

For each key the last line wins, and a line `removed` hides the work without erasing its history. The views of PollisReferences then lay your lines over Pollis's rows:

- `all_references` has every work once. A work of Pollis's you corrected appears in your version, with `origin` `corrected`; a work only you have is `user`; the rest are `pollis`.
- A work you saved before Pollis brought it records `pollis_key`: it then appears once, under Pollis's key.
- A duplicate you merged records `merged_into`; `user_aliases` follows chains of merges, and its counts and versions move to the work it was merged into.
- `all_counts` keeps every count, Pollis's and yours, each with its source and date; none is added to another.

When PollisReferences.jl corrects a work you had corrected too, your correction still wins: it is your later word on it.

9.4 Feeds you follow, and what they bring

- `~/.pollis/rss/feeds.jsonl`, the feeds you follow; for each address the last line wins:

```
{"url":"https://julialang.org/feed.xml","topic":"Julia"}
{"url":"https://www.nber.org/rss/new.xml","topic":"Economics and Central Banks","name":"NBER"}
{"url":"https://www.nber.org/rss/new.xml","removed":true}
```

  It is written once, the first time Pollis starts without it: from your old setting `pollis.rss.feeds` if you had one, else from the catalogue's defaults. A feed outside the catalogue is as welcome as one in it.
- `items/YYYY-MM.jsonl`, every item received, by month: the archive. `zotero.jsonl` and `episodes.jsonl`, the items you sent to Zotero and the episodes you downloaded.
- `~/.pollis/journals/articles.jsonl`, the article behind an email alert's item or a work saved from Literature Radar, looked up in Crossref and OpenAlex: its DOI, journal, year, citations, field-weighted citation impact and citation percentile. The view `articles` of PollisJournals gives the latest line of each, and its `journal_id` joins it to `journals`, to the Norwegian levels and to your metrics.

A new catalogue never changes what you follow: a feed it drops stays followed, and one it adds is only offered. A watcher on `feeds.jsonl` keeps every open window in step. The view `items` reads every month's file and gives each item once.

9.5 Why it holds together

- **No update writes your files.** The install road writes the packages, the copies and the databases, and nothing else.
- **No rewrite, only appends.** A crash can cut short only the last line, which is skipped; nothing earlier is ever at risk.
- **Views, not copies.** Your data is never copied into a built-in database, so there is no second copy to fall out of step, and no rebuild is needed when you add something.
- **Atomic builds.** A new database replaces the old in one rename, and a failed build keeps the old.
- **Keys join the two worlds.** ISSNs join your metrics and feeds to the journals; reference keys and DOIs join your works to Pollis's; dataset ids are table names.

9.6 Back up, move, start again

To keep everything you did, keep `~/.pollis/references`, `~/.pollis/rss`, `~/.pollis/journals/metrics`, `~/.pollis/journals/articles.jsonl` and your own databases. On another computer, put them in the same places and start the Julia REPL: the packages are installed, and the databases built over your files. Deleting `~/.pollis/databases` is always safe; it is made again at the next start.

9.7 Giving your data to everyone

A dataset, a feed or a correction that others would want can go into the package itself, through an issue or a pull request on its repository: a dataset with its source, licence and column descriptions (it must pass the licence check of `datasets.toml`); a feed with its topic and ISSNs; a correction with the source that shows it. Once merged, every Pollis has it at its next start.

10 Procedures, for maintainers

10.1 Add a dataset

1. In the GitHub clone of PollisDatasets.jl (never the installed copy), check the licence and choose the `status`; add `data/<id>.csv` and its entry in `datasets.toml`, with every column described.
2. Run the package tests; `scripts/build_data.R` rebuilds the CSV files taken from R packages.
3. Commit and push. At the next REPL start, every Pollis rebuilds PollisDatasets, and the Welcome page announces the new dataset.

10.2 A month of PollisJournals

Nothing to do: the workflow releases and commits by itself. Before editing the repository, pull, because the bot commits `Artifacts.toml`. A failed run leaves the previous release in place.

10.3 Correct a work, or add one

1. In `~/PollisReferences.jl`, edit `data/references.json`: keep it sorted by key, tab-indented, with fields in the order above. A key is given once and never recomputed, except when a correction changes the authors or the year it encodes.
2. Build Pollis against the clone and look at the differences:

```
POLLIS_REFERENCES=~/PollisReferences.jl npm run gulp compile-toml
git diff -- src/vs/workbench/contrib/model/browser/media/references/
```

3. Commit and push the package. Every Pollis gets the change at its next REPL start.
4. To bake it into the next Pollis too, write the new SHA into `build/PollisReferences.commit` and commit it with the regenerated `pollis.bib` and `references.json`.

Never remove or rename a key that any released Pollis cites. The copy would then be rejected by those Pollis versions, which would silently stay on their shipped references and miss every later correction. To retire a work, leave it in the package.

10.4 Cite a new work in a panel

Add the work to the package first, push, and bump `build/PollisReferences.commit`; only then write its key in the panel's TOML. In the other order, `compile-toml` stops with *Reference keys not in PollisReferences.jl*.

10.5 Add or correct a feed

1. In `~/PollisRSS.jl`, edit `data/feeds.json`; read the feed once to check that it answers.
2. Build against the clone, then try it in Pollis:

```
POLLIS_RSS=~/PollisRSS.jl npm run gulp compile-toml
```

3. Commit and push the package; to bake it in, bump `build/PollisRSS.commit`.

Points to watch:

- A *new note id* goes into Pollis first (a case in `catalogueNote`, localised); an older Pollis shows nothing for it, which is harmless.
- *Renaming or splitting a topic*: users who follow its feeds keep the old name in `feeds.jsonl`. `RENAMED_TOPICS` and `MOVED_FEEDS` in `rssCatalogue.ts` served a migration that ran once, on settings; a new rename needs a new migration on `feeds.jsonl`, appending a line per feed moved.
- *Changing a feed's address*: users who follow the old one keep it. Add the new one to the catalogue and decide separately whether Pollis should move followers.
- *Defaults* affect only users who start Pollis for the first time after the change.

10.6 Check a change end to end

```
source ~/.nvm/nvm.sh; nvm use 22.22.1
npm run gulp compile-toml            # the build road
npm run compile-check-ts-native      # the workbench still type-checks
julia --startup-file=no extensions/language-julia/scripts/terminalserver/install_pollisdb.jl
cmp ~/.pollis/rss/catalogue.json ~/PollisRSS.jl/data/feeds.json
cmp ~/.pollis/references/package.json ~/PollisReferences.jl/data/references.json
julia --startup-file=no -e 'include("extensions/language-julia/scripts/terminalserver/PollisDB.jl"); println(
    PollisDB.needs_install())'   # false
```

The install road sees only what is pushed to `main`.

11 Where things are

File	Role
`.../terminalserver/PollisDB.jl`	`DATA_PACKAGES`, `PACKAGE_COPIES`, `install`, the four builds, `connect`, `sql>`
`.../terminalserver/install_pollisdb.jl`	The install script
`extensions/language-julia/src/pollisDuckDB.ts`	Runs the install; copies the shipped references to `pollis.json`
`build/PollisReferences.commit`, `build/PollisRSS.commit`	The pinned commits
`build/lib/pollis-packages.ts`	`pinnedPackageFile`: clone, cache or download
`build/lib/pollis-bib.ts`	References: `pollis.bib`, `references.json`, citation copies
`build/gulpfile.toml.ts`	The `compile-toml` task
`.../model/browser/common/referenceLibrary.ts`	`pollis()`: which references the pages use
`.../monitor/browser/rssCatalogue.ts`	Catalogue access, note ids, search helpers
`.../monitor/browser/rssFeedsService.ts`	`feeds.jsonl`, migration, catalogue loading
`.../monitor/browser/journalScores.ts`	`scores.csv` and your metrics, for the RSS tags
`.../monitor/browser/articleLookup.ts`	`articles.jsonl`
`.../databases/browser/journalMetrics.ts`	Import Journal Metrics

12 Open points

- **Not yet tried in the app**: the RSS pages on `feeds.jsonl` (first-start migration, add, remove, restore, two windows) and the References pages on `package.json`.
- **Citation counts**: PollisReferences.jl has no monthly workflow that appends new counts, as PollisJournals.jl has; the counts change only by hand.
- **Default feeds**: the rules for choosing them are still to be decided.
- **Catalogue notes**: kept as ids, but displayed by no page.